
Robust Financial Reasoning in LLMs: A Controllable Benchmark with Verifiable Rewards

Alicia Larsen Victoire Laurent Aulia Kharis Rakhmasari Lara Turgut
Nino Antulov-Fantulin
Eidgenössische Technische Hochschule Zürich, Zürich

Abstract

While existing benchmarks have made substantial progress in evaluating LLMs across STEM domains, financial reasoning over structured data remains comparatively less explored. We introduce a benchmark generation framework with explicit control over reasoning depth and breadth, enabling targeted analysis of model failures on compositional numerical reasoning tasks. The framework follows a deterministic pipeline: executable computation trees are grounded in structured financial data, rendered into natural-language questions, and paired with automatically verified answers. By varying clear sources of difficulty including computation depth, expression breadth, complex financial concepts, and context, we find that open-source models can lose up to 94.4 percentage points in accuracy as reasoning depth increases, and up to 17.8 points under adversarial numerical perturbations, highlighting remaining challenges in robust financial reasoning over tables. We further show that lightweight LoRA fine-tuning on verified chain-of-thought traces recovers a substantial share of this gap, improving accuracy from 81.1% to 85.6% on held-out problems, suggesting that targeted, low-cost adaptation is a promising direction for closing the compositional reasoning gap in financial QnA.

1 Introduction

Reasoning over structured financial data remains a distinctive challenge for large language models. Unlike open-ended text generation or even mathematical problem-solving over symbolic expressions, financial reasoning demands the simultaneous satisfaction of several constraints: correct retrieval of values from structured tables, knowledge of domain-specific financial concepts and their formulas, and the ability to compose multi-step arithmetic across heterogeneous quantities such as monetary amounts, rates, and ratios.

Existing benchmarks have made substantial progress in evaluating LLMs on STEM reasoning and general numerical tasks, yet financial reasoning over structured tabular data (particularly in settings that require compositional depth and domain knowledge) remains comparatively underexplored. A core obstacle to progress in this area is the lack of benchmarks with explicit, controllable difficulty. Most existing financial QA datasets are constructed from real-world documents via human annotation or model-assisted extraction, which conflates source-level noise (including formatting irregularities, missing values, and ambiguous phrasing) with genuine failures in financial reasoning. This makes it difficult to isolate where and why models fail. Furthermore, because ground-truth answers are written manually or extracted heuristically, the cost of scaling such benchmarks to cover a broad distribution of reasoning depths is prohibitive.

We address these limitations by introducing a deterministic benchmark generation framework for financial reasoning over synthetic spreadsheets. The framework generates natural-language questions by sampling executable computation trees grounded in spreadsheet values, producing automatically verified answers without manual annotation. By controlling reasoning depth and incorporating

Work done as a project on the course “Machine Learning for Finance & Complex Systems”, Spring 2026, ETH Zurich. Students: A.L., V.L., A.K.R., L.T. Supervisor: N.A.-F. Acknowledgement: we thank Imanol Schlag and Eduard Āurech from Apertus for help and collaboration. Code available at <https://github.com/auliakharis/ML-in-Finance-and-Complex-System>

derived financial concepts such as gross profit, operating income, and net income, it enables scalable evaluation of both compositional reasoning and financial domain knowledge.

Using this framework, we construct single-turn and multi-turn benchmarks and evaluate a range of open-source LLMs across increasing reasoning complexity. We show that performance declines sharply with reasoning depth, that LoRA fine-tuning on verified chain-of-thought traces improves Qwen3-4B over its zero-shot baseline, and that OCR-style character corruption is the most effective adversarial perturbation. Together, these results provide a reproducible benchmark for studying robust compositional financial reasoning.

2 Related Work

Financial question answering benchmarks have progressively increased in complexity, from multi-step numerical reasoning over reports in FinQA (1), to conversational settings in ConvFinQA (2), to hybrid table-text evidence in TAT-QA (3), and long-context retrieval over SEC filings in FinanceBench (4). FINGPT Bench (5) offers broader financial NLP evaluation across sentiment, relation, and forecasting tasks. These datasets are constructed via human annotation, which limits scalability and makes reasoning difficulty difficult to control systematically.

General reasoning benchmarks such as DROP (6) and GSM8K (7) have been widely used to probe arithmetic and multi-step mathematical reasoning, while TabFact (8) addresses fact verification over semi-structured tables. Neither line of work targets the intersection of domain-specific financial knowledge and compositional tabular reasoning that motivates our benchmark.

Our framework addresses these gaps through a deterministic, annotation-free generation pipeline with explicit control over reasoning depth, grounded in synthetic financial spreadsheets and verified computation trees.

3 Benchmark Generation Pipeline

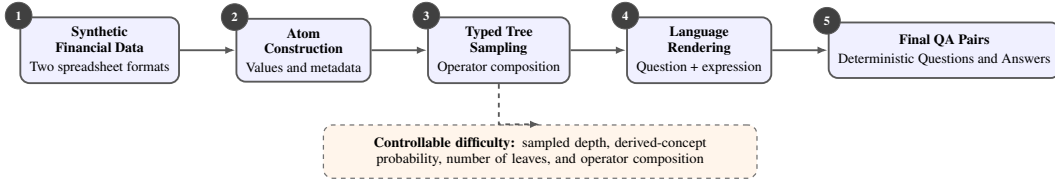


Figure 1: Benchmark generation pipeline.

We generate the benchmark using a deterministic pipeline that maps synthetic financial spreadsheets to natural-language questions with executable answers. Starting from structured financial data, the pipeline builds typed semantic representations of spreadsheet cells, samples computation graphs with controlled depth and breadth, grounds each graph in concrete values, and renders the result as a financial reasoning question. This ensures that each example has a known computation trace and an automatically verifiable answer.

3.1 Financial Data

The first stage constructs two synthetic financial data sources. The first mimics a typical 10-Q filing, with financial tables resembling those found in company reports. The second is a regularized financial sheet with consistent column names, units, and temporal structure, allowing us to isolate compositional numerical reasoning from formatting noise, extraction artifacts, and missing values. Truncated examples of both spreadsheets are shown in Table 8 and Figure 4.

The regularized sheet contains company-year observations for 15 synthetic companies from 2020 to 2025. Each row includes company metadata, such as name, ticker, sector, country, exchange, and credit rating, followed by normalized financial variables including revenue, costs, expenses, assets, liabilities, equity, cash, receivables, inventories, investments, current liabilities, capital expenditures, dividends, shares outstanding, stock price, and employees. The number of companies is adjustable; we use 15 to balance realistic report scale with manageable LLM context length.

Financial values are generated to preserve internal coherence while remaining synthetic. Revenue is initialized from company-specific ranges and evolves through sampled yearly growth or decline. Income-statement quantities are tied to revenue through plausible ratios, while balance-sheet components are sampled as percentages of related parent quantities, such as cash relative to total assets or inventories relative to cost of goods sold. Additional variables are sampled from realistic ranges or derived from intermediate quantities, yielding tables that support meaningful arithmetic and compositional reasoning.

From these spreadsheets, we construct typed financial atoms. Each atom corresponds to one spreadsheet cell and stores its numerical value together with semantic metadata: financial concept, company, fiscal year, unit, and quantity type. These atoms provide the grounding layer for question generation.

3.2 Question-Answer Pair Generation

The next stage samples a symbolic expression template, represented as a typed binary tree. Leaves are financial atoms, while internal nodes specify an operation and metadata about their children. The operator set includes addition, subtraction, multiplication, ratios, growth rates, minimum, maximum, and average. The sampler is type-aware: it only constructs expressions whose child outputs are compatible with the parent operation and each other, preventing invalid expressions such as multiplying a year by a monetary amount.

A key feature of this generation process is that because of the tree structure, the difficulty of a question can be explicitly controlled. In particular, tree depth provides a direct proxy for the number of reasoning steps required. Indeed, simple questions such as: "Provide value of company A's total assets in 2024" are represented as a simple leaf and have depth 0, while slightly more complex ones such as: "Give Maximum between company A's total assets and company B's total assets in 2024" are represented as a tree with two leaves and one operator node and thus have depth 1. Shallow expressions correspond to simple lookups or one-step arithmetic questions, while deeper expressions require the model to compose several intermediate results. The pipeline can also optionally use balanced trees, which distribute computation more evenly across branches. This allows us to also control the breadth of a question, as opposed to the depth, with a breadth to depth growth factor of 2^i for depth i . See 2 for an example of such trees.

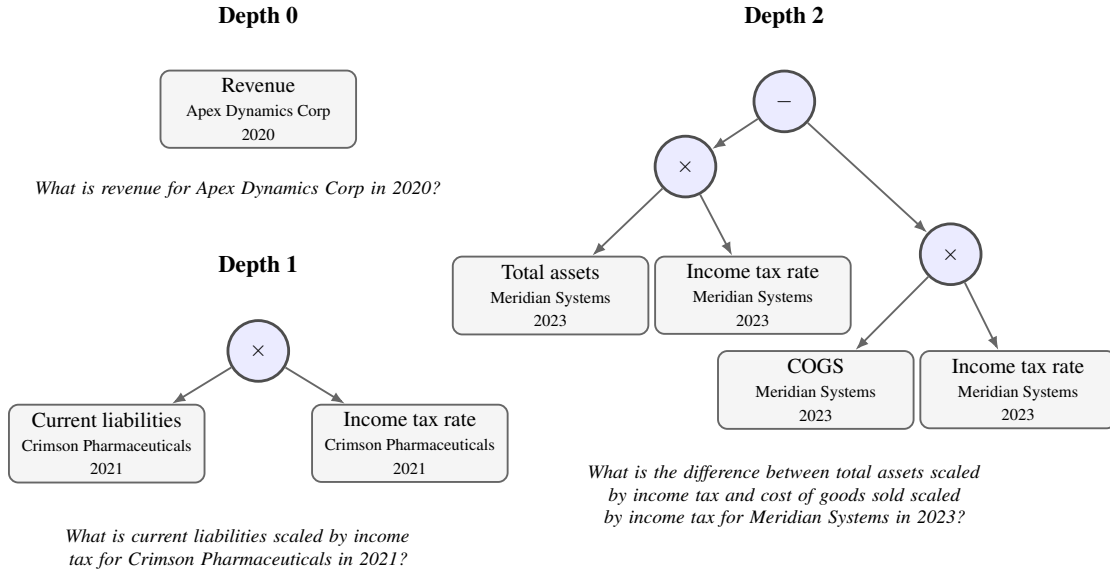


Figure 2: Examples of grounded computation trees and their rendered natural-language questions. Leaves include spreadsheet metadata such as concept, company, and year; internal nodes define the arithmetic computation.

The pipeline also supports named derived financial concepts, such as gross profit, operating income, pretax income, income tax expense, net income, current assets, long-term assets, and long-term liabilities. These concepts are stored as special atoms with hidden executable formula trees. Because

their formulas are not shown in the question and the values cannot be directly looked up, they test financial knowledge in addition to arithmetic reasoning.

To avoid ambiguity between named financial concepts and arbitrary equivalent arithmetic expressions, the sampler applies a rejection step for protected derived concepts. If a sampled template reconstructs the canonical formula of a protected concept without explicitly using the corresponding named concept atom, the tree is rejected and resampled. This prevents, for example, a manually sampled difference between revenue and cost of goods sold from being treated differently than the named concept "gross profit". As a result, the benchmark maintains a clear distinction between raw arithmetic composition and explicitly introduced financial abstractions.

The sampling process is parameterized to control the distribution of generated examples. In particular, we specify a minimum and maximum template depth, and each question is sampled with a depth drawn from this range. This makes it possible to generate mixtures of simple and highly compositional questions, or to focus evaluation on a specific difficulty band. We also control the probability of sampling named derived concepts, in order to ensure that they appear more often than regular spreadsheet cells.

After a valid template is sampled, the pipeline grounds it in concrete spreadsheet atoms using a binding environment that tracks constraints such as company, fiscal year, and financial concept. Same-period operations bind operands to compatible company-year contexts. Growth operations select two distinct years for the same company and concept, while temporal aggregations such as minimum, maximum, and average select a set of reporting years for a given company-concept pair. The result is a fully bound expression whose leaves are spreadsheet cells and whose internal nodes define the exact computation.

The bound expression is then rendered into a natural-language question through a bottom-up semantic procedure. Leaves are translated into short phrases using their metadata, such as "*revenue for Apex Dynamics Corp in 2020*". Internal nodes then combine their children with operator-specific templates; for example, a subtraction node over revenue and cost of goods sold becomes "*the difference between revenue and cost of goods sold for Apex Dynamics Corp in 2020*". Because intermediate phrases retain metadata such as company, year, unit, and concept, the renderer can avoid redundant phrasing when context is shared across subexpressions.

After rendering, we apply lightweight linguistic data augmentation to the natural-language questions. These augmentations preserve the underlying computation graph and ground-truth answer, but introduce surface-level variation in how the question is phrased. For example, we introduce minor typographical errors, vary capitalization, and replace standard phrasing with informal alternatives such as "*What is the value of ...*" versus "*How would you calculate ...*". Such perturbations enable the assessment of model robustness to superficial linguistic variations and reduce the likelihood that benchmark performance is driven by sensitivity to specific prompt templates. Evaluating model behavior across multiple semantically equivalent formulations provides a more reliable measure of financial reasoning and domain knowledge, while improving the ecological validity of the benchmark by more closely reflecting the diversity of expressions observed in practical financial applications.

The executable expression linked to the rendered question is evaluated directly to obtain the ground-truth answer. Since the answer is computed from the expression tree rather than written manually, each question-answer pair is automatically verified. For every generated example, we store the natural-language question, the numeric answer, the full symbolic expression, the template expression, the requested and realized depths, the random seeds used for template sampling and binding, the probability of sampling derived concepts, and the list of spreadsheet cells used as leaves. This metadata makes the benchmark reproducible and enables fine-grained analysis by depth, operator type, number of required cells, use of derived concepts, and temporal reasoning structure.

4 Extensions

4.1 Multi-Turn Extension

Single-turn financial QA conflates two distinct failure modes: errors in intermediate arithmetic and errors in compositional planning. To probe the model behavior under sequential reasoning, we extend the existing one-shot benchmark into a multi-turn format by exploiting the symbolic expression

trees already annotated in the source data. The construction is fully deterministic and introduces no additional human labeling.

Let each problem be associated with an expression tree T of bound depth $d(T)$, where internal nodes correspond to parenthesized sub-expressions in the string representation form of T . We convert T into a multi-turn dialogue under the following rules:

1. **Eligibility.** Only problems with $d(T) \geq 1$ are converted; shallower trees collapse to the original single-turn item and are retained unchanged.
2. **Turn segmentation.** Each parenthesized sub-expression in T maps to exactly one turn. Turns are ordered by computational dependency, so that each turn depends only on values produced in previous turns.
3. **Terminal turn.** The final turn restates the original question in full, ensuring that the multi-turn variant remains directly comparable to its one-shot counterpart.
4. **Ground truth.** Per-turn targets are read directly from the corresponding nodes of T , yielding exact, tree-aligned supervision at every step without additional annotation.

Because the procedure is purely structural, the multi-turn benchmark inherits the scale and coverage of the single-turn benchmark while exposing intermediate computations as first-class evaluation targets. This enables per-depth diagnostics that are not identifiable from single-question correctness.

Original question (single-turn). What is the operating income for NovaStar Technologies in 2024?
Expression tree. `subtract(subtract(revenue, cogs), operating_expenses)` ($d(T) = 2$)

Turn 1 — User: What is the gross profit for NovaStar Technologies in 2024?
Turn 1 — Model: \$100,535
Turn 2 — User: What is the operating income for NovaStar Technologies in 2024?
Turn 2 — Model: \$80,626

Figure 3: Multi-turn conversion of a depth-2 problem.

4.2 Adversarial Robustness

A deployed financial QA system encounters tables that are noisy, partially missing, or assembled from heterogeneous sources. To probe model robustness under these conditions, we construct a small adversarial benchmark by perturbing the input tables of existing items while leaving the question and ground-truth answer unchanged. All perturbations are restricted to *question-irrelevant fields*, i.e., cells that do not appear in the expression tree used to compute the answer; this guarantees that the ground truth is invariant under attack and that any drop in accuracy reflects a model failure rather than a label shift.

We consider four families of perturbations, chosen to mirror failure modes documented in real financial pipelines (OCR ingestion, multi-sheet consolidation, manual data entry):

(i) Missing values. For each item, we sample a random subset of question-irrelevant cells and replace their contents with an empty value. This simulates incomplete filings and partially extracted tables.

(ii) Garbage values. Selected question-irrelevant cells are overwritten with extreme or out-of-distribution values drawn from $\{-10^7, 10^{12}, -1, \text{"ERROR"}\}$. This probes whether the model is anchored to the relevant cells or distracted by salient outliers elsewhere in the table.

(iii) OCR look-alike substitutions. Digits in question-irrelevant numerical cells are replaced with visually similar non-digit characters drawn from the substitution map $\{ "0":["0", "o"], "1":["1", "l"], "2":["2", "Z"], "5":["5", "S"], "6":["6", "b"], "8":["8", "B"], "9":["9", "g"] \}$. This reproduces a common failure mode of OCR pipelines used to convert scanned financial statements into CSV.

(iv) Cross-sheet contamination. Question-irrelevant rows are replaced with rows sampled from financial statements of *other* companies in the corpus. The resulting table is internally well-formed but semantically inconsistent, mimicking errors in multi-document consolidation pipelines.

Because all four perturbations preserve the ground-truth answer by construction, accuracy on the perturbed benchmark is directly comparable to accuracy on the clean benchmark, and the gap isolates the model’s sensitivity to each noise type.

In addition to tabular noise, we introduce a perturbation at the question level. Specifically, we append irrelevant contextual information to the question, such as rumors, general market sentiment, or statements about other companies. This tests whether models can distinguish the information required for the computation from distracting but superficially plausible context.

4.3 Controlling Reasoning Difficulty

Beyond robustness to noise, the synthetic generation process allows for systematic control of the intrinsic difficulty of a question. Since each question is derived from an expression tree over table cells, it was possible to vary both the structure of the computation and the numerical properties of the underlying values while preserving full control over the ground-truth answer.

(i) Reasoning depth. The depth of the generated expression tree determines the number of compositional reasoning steps required to obtain the answer. By increasing the maximum tree depth during generation, we obtain questions that place progressively greater demands on multi-step numerical reasoning and intermediate-state tracking.

(ii) Reasoning breadth. Reasoning breadth controls how many intermediate quantities must be computed and combined within a single question. We operationalize this through a balanced-tree setting, where multiple branches of the expression tree have comparable depth rather than forming a mostly linear chain. This requires the model to compute several sub-results before producing the final answer. As a result, broader questions increase the number of relevant table entries and intermediate values that must be tracked simultaneously, testing whether models can correctly retrieve, compute, and compose multiple pieces of financial information.

(iii) Value scaling. Numerical magnitude is controlled through a multiplicative scaling factor applied during data generation. Larger scaling factors produce larger operands without changing the underlying reasoning structure, allowing numerical difficulty to be varied independently of reasoning depth.

4.4 LoRA

To improve the model’s ability to perform the domain-specific numerical reasoning and calculation tasks required by our financial benchmark, a pretrained model can be fine-tuned using Low-Rank Adaptation (LoRA). Rather than updating the full set of model parameters, LoRA introduces trainable low-rank matrices while keeping the original model weights frozen. In our implementation, LoRA was applied to the attention projection layers (`q_proj`, `k_proj`, `v_proj`, and `o_proj`) as well as the feed-forward network projections (`gate_proj`, `up_proj`, and `down_proj`). Adapting the attention layers allows the model to learn task-specific patterns in how financial information is represented and attended to, while modifying the feed-forward projections enables adjustments to the model’s internal feature transformations and reasoning capabilities. A rank of ($r = 16$), a scaling factor ($\alpha = 32$), and a dropout rate of 0.05 were used, to provide a balance between adaptation capacity and parameter efficiency. This configuration substantially reduced the number of trainable parameters and the associated computational requirements while enabling effective specialization of the base model for financial reasoning tasks.

5 Results

5.1 Benchmarking results

5.1.1 Small-Model Benchmarking

We evaluate three open small models : Gemma-4 4B-it, Qwen3.5-4B, and Qwen3.5-9B on three task settings: single-turn questions over raw 10-Q filings, single-turn questions over simplified financial sheets, and the multi-turn extension introduced in Section 4.1. Results are reported in Table 1.

Three observations follow. First, Qwen3.5-4B is the strongest model in every setting and outperforms the larger Qwen3.5-9B by 8.3 percentage points on simplified sheets and 14.4 points on 10-Q filings, suggesting that scale alone does not determine performance on this benchmark. Second, multi-turn performance is notably uneven across models: Qwen3.5-4B maintains strong accuracy in the multi-turn setting (82.2 on simplified sheets, 92.2 on 10-Q filings), whereas Gemma-4 4B-it drops sharply

Model	Simplified sheet			10-Q filing		
	Single-turn	Multi-turn	Overall	Single-turn	Multi-turn	Overall
Gemma-4 4B-it	73.3	55.6	64.5	52.2	86.7	69.5
Qwen3.5-4B	88.9	82.2	85.6	78.9	92.2	85.6
Qwen3.5-9B	85.6	68.9	77.3	55.6	86.7	71.2

Table 1: Accuracy (%) of small open models across task settings.

to 55.6 on simplified sheets despite reasonable single-turn scores. Third, all three models exceed 50% overall accuracy, establishing that the benchmark is tractable for the small-model regime while leaving substantial headroom, particularly on the 10-Q and multi-turn settings.

Table 2 reports per-depth accuracy on the multi-turn benchmark for simplified financial sheets. Sample sizes drop sharply at higher depths, so results for turns ≥ 4 should be read as indicative rather than conclusive. For 10-Q filings, the same sparsity arises earlier, and results are reported only up to depth 4.

Model	Simplified							10-Q			
	1	2	3	4	5	6	12	1	2	3	4
	(n=51)	(n=12)	(n=8)	(n=6)	(n=7)	(n=5)	(n=1)	(n=51)	(n=19)	(n=8)	(n=12)
Gemma-3 4B-it	72	75	12	33	14	0	0	86.3	94.7	87.5	75.0
Qwen3.5-4B	88	92	75	67	57	60	100	100	78.9	87.5	83.3
Qwen3.5-9B	74	100	38	67	57	20	0	84.3	94.7	87.5	83.3

Table 2: Accuracy (%) by number of turns on simplified financial sheets and 10-Q filings. Sample sizes are shown underneath each turn count; 10-Q results are reported up to four turns due to sparse sample sizes at higher turn counts.

Two patterns emerge from the simplified sheet results. Qwen3.5-4B degrades most gracefully with depth and retains a clear lead at turns ≥ 3 , where Gemma-3 collapses to near-zero accuracy and Qwen3.5-9B becomes unstable. At intermediate depths (4–5 turns), Qwen3.5-4B and Qwen3.5-9B perform comparably, so the overall gap between the two Qwen variants is driven primarily by the shallow and deep ends of the distribution. The thin tail (turns ≥ 4) is a limitation of the current benchmark; we note it as a target for future expansion.

On 10-Q filings, accuracy is consistently higher and degrades less with depth than on simplified sheets, with Gemma-3 and Qwen3.5-9B performing comparably across all reported depths. The absence of a clear depth-accuracy trend suggests that turn count is a less reliable proxy for difficulty when the underlying document structure is irregular.

5.1.2 Large-Model Benchmarking

We evaluate five open large models — Apertus-70B, Qwen3.5-Coder-30B, Llama-3.3-70B, GPT-OSS-120B, and DeepSeek-Coder-33B — under the same three settings as the small-model evaluation. Results are reported in Table 3, with single-turn vs. multi-turn deltas shown in the final column.

Model	Simplified				10-Q			
	Single-turn	Multi-turn	Overall	Δ	Single-turn	Multi-turn	Overall	Δ
Apertus-70B	82	64	73	−17.8	52.2	91.1	71.7	+38.9
Qwen3.5-Coder-30B	83	78	81	−5.5	73.3	91.1	82.2	+17.8
Llama-3.3-70B	91	82	87	−8.9	83.3	96.7	90.0	+13.4
GPT-OSS-120B	86	93	90	+7.7	—	—	—	—
DeepSeek-Coder-33B	57	29	43	−27.8	—	—	—	—

Table 3: Accuracy (%) of large open models across task settings. Δ denotes the change from single-turn to multi-turn, in percentage points.

Three observations stand out. First, parameter count is a poor predictor of performance in this regime: Qwen3.5-Coder-30B (81% overall on simplified sheets) outperforms Apertus-70B (73%) at less than half the parameter count, and DeepSeek-Coder-33B trails the field on simplified sheets

despite a comparable size. Second, Llama-3.3-70B achieves the highest overall scores across both task types, leading on simplified single-turn (91%) and reaching 90.0% overall on 10-Q filings, with a modest 8.9-point single-to-multi-turn drop on simplified sheets. Third, on simplified sheet, GPT-OSS-120B is the only model whose accuracy *improves* from single-turn to multi-turn (+7.7 points on simplified sheets), suggesting that the per-turn decomposition acts as a scaffold that benefits some models while penalizing others; this asymmetry is one of the more interesting findings the multi-turn extension surfaces. Note that 10-Q results for GPT-OSS-120B and DeepSeek-Coder-33B are currently unavailable (indicated by ‘–’ in Table 3) and will be reported in a future revision.

5.2 Adversarial Robustness Results

We evaluate Qwen3.5-27B on a combined adversarial benchmark of 90 items per attack family. In addition to the four perturbations defined in Section 4.2, the combined setting also applies a uniform $\times 10^6$ scale to numerical fields, injects question-irrelevant auxiliary text into the prompt, and embeds prompt-injection commands inside the table itself. Per-attack accuracy is reported in Table 4.

Attack	10-Q		Simplified	
	Correct / Total	Accuracy (%)	Correct / Total	Accuracy (%)
Baseline (clean)	81 / 90	90.0	87 / 90	96.7
Missing values	81 / 90	90.0	71 / 90	78.9
Garbage values	77 / 90	85.6	64 / 90	71.1
Look-alike substitution	65 / 90	72.2	71 / 90	78.9
Cross-sheet contamination	73 / 90	81.1	72 / 90	80.0
Combined	69 / 90	76.7	67 / 90	74.4
Overall	446 / 540	82.6	432 / 540	80.0

Table 4: Qwen3.5-27B accuracy under each adversarial perturbation across 10-Q and simplified settings. All attacks are applied only to question-irrelevant fields, so the ground truth is invariant, except for the big-numbers scaling attack, where the ground truth is rescaled uniformly across all samples.

The model is fully robust to missing values (no accuracy drop) and only mildly affected by garbage values (−4.4pp) and cross-sheet contamination (−8.9pp), indicating that the model attends primarily to the question-relevant cells and ignores irrelevant numeric noise. OCR look-alike substitution is by far the most damaging single attack (−17.8pp), suggesting that character-level corruption of numerals disrupts the model’s value extraction more than semantic noise does. The combined setting drops a further 5.5 points below the look-alike attack alone, indicating that the four perturbations are not fully redundant but also not strictly additive.

5.3 Controlled Reasoning Difficulty Results

To investigate the limitations of Llama3.5-4B on arithmetic reasoning tasks, we evaluated performance as a function of (i) reasoning depth and (ii) the magnitude of the multiplication factor. Results are summarized in Tables 5 and 6.

Depth	1	2	3	4	5	6	7	8	9	10
Accuracy (%)	94.4	–	60.0	60.0	54.4	43.3	16.7	5.7	2.2	0.0

Table 5: Accuracy as a function of reasoning depth, fixing the multiplication factor to 1.

Multiplication Factor	10^0	10^3	10^4	10^5	10^6	10^9
Accuracy (%)	94.4	78.9	80.0	75.6	67.8	60.0

Table 6: Accuracy as a function of multiplication factor while keeping reasoning depth fixed at 1.

The results reveal that reasoning depth is the dominant source of difficulty. Accuracy drops from 94.4% at depth 1 to around 60% at depths 3-4, then declines rapidly to 2.2% at depth 9 and 0% at depth 10, indicating that errors accumulate over long reasoning chains.

Increasing the multiplication factor also degrades performance, but more gradually. Accuracy decreases from 94.4% for a factor of 1 to 67.8% at 10^6 and 60.0% at 10^9 , suggesting that large numerical values are challenging but less so than increased reasoning depth.

Combining both factors further amplifies the difficulty. With depth $d = 5$ and multiplication factor $\text{mf} = 10^6$, accuracy falls to 26.7%, substantially below the results obtained when varying either parameter independently. Overall, these findings indicate that reasoning depth is the primary limitation, while large numerical magnitudes further exacerbate performance degradation.

5.4 LoRA Fine-Tuning with Verified Chain-of-Thought

We fine-tune Qwen3.5-4B on the financial QA training pool using a two-phase recipe. **Phase 1** generates chain-of-thought (CoT) traces for each training item and retains only those whose final answer matches the ground-truth value derived from the expression tree, yielding 608 verified examples from a pool of 688 (88% retention); Table 9 shows the depth distribution before and after verification, indicating that verification success degrades gracefully with reasoning depth. **Phase 2** fine-tunes the model using LoRA adapters (0.50% trainable parameters) on the verified traces, with hyperparameters summarized in Table 10. We sweep r , α , and dropout on the held-out benchmark ($n=90$; Table 7): $r=8$, $\alpha=8$, dropout 0.10 attains the highest accuracy (85.6%), tied at dropout 0.15. Accuracy is most sensitive to the α/r ratio, with $\alpha/r \neq 1$ erasing the gain over baseline.

r	α	Dropout	Accuracy
Baseline (no LoRA)			81.1% (73/90)
8	8	0.10	85.6% (77/90)
8	8	0.15	85.6% (77/90)
8	4	0.10	83.3% (75/90)
16	32	0.05	83.3% (75/90)
4	4	0.10	82.2% (74/90)
8	16	0.10	81.1% (73/90)
8	16	0.15	81.1% (73/90)
8	4	0.15	81.1% (73/90)

Table 7: LoRA hyperparameter sweep on the held-out benchmark ($n=90$). Baseline is the zero-shot base model without LoRA adapters. We vary rank $r \in \{4, 8, 16\}$, scaling factor $\alpha \in \{4, 8, 16, 32\}$, and dropout $\in \{0.05, 0.10, 0.15\}$.

6 Conclusion

We introduced a deterministic benchmark generation framework for evaluating financial reasoning in large language models over structured tabular data. By generating executable computation trees grounded in synthetic financial spreadsheets, our approach produces automatically verified question-answer pairs while providing explicit control over reasoning depth, expression breadth, financial-domain complexity, and linguistic variation. This enables systematic evaluation of compositional financial reasoning without the scalability and annotation limitations of existing benchmarks.

Our experiments demonstrate that reasoning depth is the primary factor limiting model performance, with accuracy degrading substantially as the number of compositional reasoning steps increases. While LoRA fine-tuning on verified chain-of-thought traces improves performance over strong zero-shot baselines, considerable headroom remains, particularly on deeper reasoning tasks. We further show that OCR-style character corruption poses a greater challenge than missing or irrelevant data, highlighting the importance of robust numerical extraction in practical financial applications. The proposed multi-turn benchmark additionally provides fine-grained insight into intermediate reasoning failures that are not observable through end-to-end evaluation alone.

Overall, the benchmark provides a reproducible and extensively tested metric for studying financial reasoning, model robustness, and reasoning scalability in LLMs. Future work will expand the diversity of financial concepts and document formats, increase coverage of deeper reasoning chains, incorporate cross-table and cross-document reasoning, and explore verifier-guided training and inference methods that further improve reliability on complex financial analysis tasks.

References

- [1] Zhiyu Chen, Wenhui Chen, Charese Shen, Oliver Kroeger, Winnie Au, Bhavana Iyer, Hannah Jo, Elise Liu, Matt Gardner, and William Yang Wang. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of EMNLP*, 2021.
- [2] Zhiyu Chen, Shiyang Li, Charese Shen, Xinyi Feng, Xifeng Yan, and William Yang Wang. ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of EMNLP*, 2022.
- [3] Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of ACL*, 2021.
- [4] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. FinanceBench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.
- [5] Yangyang Yu, Haohang Li, Zhuohan Gu, Yupeng Cao, Kai Yan, Chenyu Shi, Zining Zhu, Koduvayur Subbalakshmi, Guojun Xiong, Jimin Huang, Lingfei Qian, Xueqing Peng, and Qianqian Xie. FinGPT-Bench: Evaluating ChatGPT and efficient fine-tuning of open-source LLMs for financial natural language processing. *arXiv preprint arXiv:2306.06031*, 2023.
- [6] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of NAACL*, 2019.
- [7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [8] Wenhui Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyu Zhou, and William Yang Wang. TabFact: A large-scale dataset for table-based fact verification. In *Proceedings of ICLR*, 2020.

A Appendix

A.1 Data Sources

Tables 8 and Figure 4 illustrate the two tabular data sources used in our benchmark: synthetic financial spreadsheets covering the simplified setting, and truncated 10-Q filings covering the realistic setting.

Company	Ticker	Sector	Year	Revenue	COGS	Assets	Income Tax
Apex Dynamics Corp	APEX	Technology	2020	67,781	27,804	71,017	0.2481
Apex Dynamics Corp	APEX	Technology	2021	...	...	...	...
Meridian Systems Inc	MSYS	Technology	2020	...	...	...	...
...							

Table 8: Truncated example of a synthetic financial spreadsheet used as the tabular data source in the simplified setting.

CONDENSED CONSOLIDATED STATEMENT OF OPERATIONS				
	Q Oct 2025	Q Oct 2024	YTD Oct 2025	YTD Oct 2024
Revenues:				
Net sales	98,077	104,345	294,231	313,035
Membership and other income	527	561	1,581	1,683
Total revenues	98,604	104,906	295,812	314,718
Costs and expenses:				
Cost of sales	75,576	80,406	226,728	241,218
Research and development	526	560	1,578	1,680
Selling, general and administrative	20,523	21,835	61,569	65,505
Total costs and expenses	96,625	102,801	289,875	308,403

Figure 4: Truncated example of a synthetic 10-Q sheet used as the tabular data source in the realistic setting.

A.2 Training Details

Table 9 reports the depth distribution of verified CoT traces, and Table 10 lists the Phase-2 LoRA fine-tuning hyperparameters.

Setting	Subset	0	1	2	3	4	5	6	7	8	9
Simplified	Pool ($n=688$)	150	203	135	63	56	46	23	9	2	1
Simplified	Verified ($n=608$)	146	191	116	54	41	41	16	3	0	0

Table 9: Depth distribution of the training pool and verified CoT subset retained for fine-tuning in the simplified setting.

Hyperparameter	Value
Base model	Qwen3-4B
LoRA trainable parameters	0.50%
Epochs	3
Effective batch size	16 (per-device $1 \times \text{grad. accum.}$ 16)
Learning rate	2×10^{-4} , cosine schedule, 5% warmup
Optimizer	paged AdamW (8-bit)
Precision	bf16
Max sequence length	2560

Table 10: Phase-2 LoRA fine-tuning hyperparameters.